Universidad Nacional de Colombia

Facultad de Ingeniería y Adminitración

Inteligencia Artificial para la Agroindustria

Clasificación con Redes Neuronales Profundas

Componente Tabular (MLP) y Componente de Imágenes (CNN)

Basado en *An Introduction to Statistical Learning with Applications in Python* [James et al., 2023]

Estudiante de Ingeniría Agronómica

Cristian Andres Bravo Narvaez

cbravon@unal.edu.co

Docente

John Jairo Leal Gomez

Facultad de Ingeniería y Administración

April 28, 2026

Abstract

Este informe describe la implementación y evaluación de dos arquitecturas de redes neuronales profundas para problemas de clasificación supervisada: un Perceptrón Multicapa (MLP) aplicado al conjunto de datos tabulares `phpnThNfi.arff` (4 839 instancias, 2 clases), y una Red Neuronal Convolucional (CNN) aplicada al dataset `potato_subset` (456 imágenes RGB, 3 clases de enfermedades en papa). Ambos modelos fueron implementados en Python con PyTorch, siguiendo los principios metodológicos del libro *An Introduction to Statistical Learning with Applications in Python* [James et al., 2023]. El MLP alcanzó una *accuracy* del **98.86%** y la CNN del **94.57%**. Se analizan las decisiones de diseño de la arquitectura, el preprocesamiento de datos, las estrategias de regularización y las métricas de evaluación, conectando cada decisión con los temas estudiados en el curso.

Contents

1 Introducción **4**

2 Temas Estudiados: Marco Teórico **4**
2.1 Capítulo 2 – Aprendizaje Estadístico . . . 4
2.1.1 ¿Qué es el aprendizaje estadístico? . . . 4
2.1.2 ¿Por qué estimar f? . . . 5
2.1.3 ¿Cómo estimamos f? . . . 5
2.1.4 Compromiso Sesgo-Varianza . . . 6
2.2 Capítulo 4 – Clasificación . . . 6
2.2.1 Regresión Logística . . . 6
2.2.2 Modelo Logístico . . . 6
2.2.3 Función de Pérdida: Entropía Cruzada . . . 7
2.2.4 Regresión Logística Multinomial . . . 7
2.2.5 Métricas de Evaluación . . . 7
2.3 Capítulo 8 – Árboles y Métodos de Conjunto . . . 7
2.3.1 Árboles de Decisión – Predicción por Estratificación . . . 7
2.3.2 Random Forest y Bosques Aleatorios . . . 8
2.4 Capítulo 10 – Aprendizaje Profundo . . . 8
2.4.1 Red Neuronal Multicapa (MLP) . . . 8
2.4.2 Regularización por Dropout . . . 9
2.4.3 Red Neuronal Convolucional (CNN) . . . 9

3 Metodología: Componente Tabular (MLP) **10**
3.1 Carga y exploración de datos . . . 10
3.2 Preprocesamiento . . . 10
3.3 Arquitectura MLP . . . 11
3.4 Entrenamiento . . . 11

4 Metodología: Componente de Imágenes (CNN) **11**
4.1 Carga del dataset . . . 11
4.2 Transformaciones y aumento de datos . . . 12
4.3 Arquitectura CNN . . . 12
4.4 Entrenamiento . . . 12

5 Resultados: Componente Tabular (MLP) **12**
5.1 Curva de pérdida . . . 12
5.2 Métricas de evaluación . . . 13

6 Resultados: Componente de Imágenes (CNN) **13**
6.1 Curva de pérdida . . . 13
6.2 Accuracy y matriz de confusión . . . 14

7 Análisis y Conclusiones **15**
7.1 Análisis del rendimiento . . . 15
7.2 Limitaciones y trabajo futuro . . . 15

7.3 Conclusiones 16

1 Introducción

El presente proyecto final integra los conceptos teóricos y prácticos estudiados a lo largo del curso de Inteligencia Artificial, cubriendo desde los fundamentos del aprendizaje estadístico supervisado hasta la implementación de redes neuronales profundas. El trabajo se organiza en dos componentes complementarios:

Componente Tabular. Se trabaja con el archivo `phpnThNfi.arff`, un conjunto de datos en formato ARFF (*Attribute-Relation File Format*) compuesto por 4 839 registros con cinco variables numéricas de entrada ($V_1, \ldots, V_5$) y una variable objetivo binaria (`Class` $\in \{1, 2\}$). La distribución de clases es marcadamente desbalanceada: 94.6% clase 1 (916 instancias en test) frente al 5.4% clase 2 (52 instancias en test). La tarea consiste en desarrollar un modelo de clasificación empleando una Red Neuronal Multicapa (MLP).

Componente de Imágenes. Se emplea el dataset `potato_subset`, que contiene 456 imágenes RGB de hojas de papa clasificadas en tres categorías: *Early Blight* (tizón temprano), *Late Blight* (tizón tardío) y *Healthy* (saludable). El subconjunto proviene de un conjunto más grande de imágenes de enfermedades foliares. La tarea es desarrollar un modelo de clasificación mediante una Red Neuronal Convolucional (CNN).

Ambos modelos fueron implementados en Python con PyTorch, entregados en GitHub y documentados en este informe compilado en LaTeX, tal como se indicó en los requerimientos del proyecto: comprensión del problema, explicación matemática del método, explicación conceptual del funcionamiento, ejemplo de apoyo, implementación con PyTorch y evaluación de resultados.

2 Temas Estudiados: Marco Teórico

2.1 Capítulo 2 – Aprendizaje Estadístico

2.1.1 ¿Qué es el aprendizaje estadístico?

El aprendizaje estadístico es un conjunto de enfoques para estimar f (modelo predictivo), que engloba el aprendizaje automático supervisado [James et al., 2023, Cap. 2]. La motivación fundamental proviene de la relación entre variables predictoras y una respuesta. Por ejemplo, la asociación entre publicidad (TV, radio, periódico) y ventas en 200 mercados:

$$Y = f(\mathbf{X}) + \varepsilon, \qquad \mathbf{X} = (X_1, X_2, \ldots, X_p), \tag{1}$$

donde f es la función desconocida a estimar, ε es el error aleatorio con $\bar{\varepsilon} \approx 0$, e Y es la respuesta. El aprendizaje estadístico proporciona el conjunto de enfoques para estimar f (modelo predictivo), siendo el ML supervisado el más relevante para este proyecto.

Aprendizaje supervisado. Para cada observación de las mediciones predictoras X_i, $i = 1, \ldots, n$, existe una medición de respuesta asociada Y_i. El modelo relaciona la respuesta con los predictores con el objetivo de predecir la respuesta para futuras observaciones. Los métodos clásicos incluyen Regresión Lineal y Regresión Logística, así como enfoques

modernos como GAM (Modelos Aditivos Generalizados), Boosting y SVM (Máquinas de Vectores de Soporte).

2.1.2 ¿Por qué estimar f?

Existen dos razones principales para estimar f: predicción e inferencia [James et al., 2023, Sec. 2.1].

Predicción. Cuando el conjunto de entrada X produce una salida Y que no se obtiene con facilidad, se estima $\hat{f}$ tal que:

$$\hat{Y} = \hat{f}(\mathbf{X}). \tag{2}$$

La precisión de $\hat{Y}$ como predicción de Y depende del *error reducible* (que podemos minimizar mediante mejores técnicas de aprendizaje) y del *error irreducible* (inherente al problema: $Y = f(\mathbf{X}) + \varepsilon$). Por tanto:

$$E(Y - \hat{Y})^2 = \underbrace{[f(\mathbf{X}) - \hat{f}(\mathbf{X})]^2}_{\text{Error reducible}} + \underbrace{\mathrm{Var}(\varepsilon)}_{\text{Error irreducible}} . \tag{3}$$

Inferencia. La necesidad de conocer la relación entre Y y $X_1, X_2, \ldots, X_p$, estimando f no para predicciones sino para entender: ¿qué predictores están asociados a la respuesta?, ¿cuál es la relación entre la respuesta y cada predictor?, ¿puede modelarse la relación entre Y y X mediante una ecuación lineal?

2.1.3 ¿Cómo estimamos f?

Se asume que hemos observado un conjunto de n puntos de datos distintos (datos de entrenamiento), que usamos para entrenar o enseñar a nuestro método a estimar f [James et al., 2023, Sec. 2.1.3].

Métodos paramétricos. Enfoque basado en dos pasos:

1. Se hace una suposición sobre la forma funcional de f. Por ejemplo, una suposición muy simple es que f es lineal en X:

   $$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p. \tag{4}$$

   En lugar de estimar una función arbitraria de p dimensiones, solo necesitamos estimar los $p + 1$ coeficientes.

2. Después de seleccionar un modelo, necesitamos ajustar o entrenar el modelo. En el caso del modelo lineal, el enfoque más común se denomina mínimos cuadrados ordinarios.

El modelo paramétrico puede no coincidir con la verdadera forma desconocida de f. Los modelos flexibles requieren más parámetros y pueden conducir a sobreajuste (*overfitting*).

Métodos no paramétricos. No hacen suposiciones explícitas sobre la forma funcional de f; buscan una estimación de f lo más cerca posible a los datos sin irregularidades. Requieren un número grande de observaciones. Por ejemplo, *thin-plate spline* produce una estimación lo más cerca posible a los datos observados, sujeta a un ajuste suave.

2.1.4 Compromiso Sesgo-Varianza

El error de predicción esperado puede descomponerse en tres componentes fundamentales [James et al., 2023, Sec. 2.2]:

$$\mathbb{E}\big[(Y - \hat{f}(\mathbf{X}))^2\big] = \underbrace{\big[\text{Sesgo}(\hat{f}(\mathbf{X}))\big]^2}_{\text{Sesgo}^2} + \underbrace{\text{Var}\big(\hat{f}(\mathbf{X})\big)}_{\text{Varianza}} + \underbrace{\text{Var}(\varepsilon)}_{\text{Error irreducible}} . \tag{5}$$

La forma en U observada en el MSE de prueba resulta de estas dos propiedades en competencia. Los métodos más flexibles reducen el error de entrenamiento pero pueden aumentar el error de prueba (*curva en V*). Este es el **criterio de flexibilidad** clave en la selección de modelos.

El MSE se calcula usando los datos de entrenamiento con los que se ajustó el modelo (MSE de entrenamiento). Sin embargo, nos interesa realmente cuán bien funciona el método sobre datos de entrenamiento –precisión de predicciones con datos de prueba: $\text{Ave}(y_0 - \hat{f}(x_0))^2$, donde (x_0, y_0) es una observación de prueba no utilizada en el experimento.

Cuando un método tiene bajo MSE de entrenamiento pero alto MSE de prueba, se dice que está **sobreajustado** a los datos. La solución es la **Validación Cruzada**.

2.2 Capítulo 4 – Clasificación

2.2.1 Regresión Logística

La regresión logística modela la probabilidad de que Y pertenezca a una categoría particular. Para el conjunto de datos Default, la respuesta cae en una de dos categorías: Sí o No [James et al., 2023, Cap. 4].

Si la probabilidad de incumplimiento dado el saldo puede escribirse como $\Pr(\textit{default} = \textit{Sí} \mid \textit{balance})$, se abrevia como $p(\textit{balance}) = 0 \leq x \leq 1$. Si $p(\textit{balance}) \geq 0.5$, se predice $\textit{default} = \text{Sí}$.

2.2.2 Modelo Logístico

Modelar $p(X) = \Pr(Y = 1 \mid X)$ usando la función logística:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}, \tag{6}$$

lo que garantiza que las probabilidades estén siempre en $[0, 1]$. Reorganizando la ecuación (6):

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X. \tag{7}$$

El término de la izquierda se llama *log-odds* o *logit*. Los coeficientes se estiman por máxima verosimilitud (*maximum likelihood*).

2.2.3 Función de Pérdida: Entropía Cruzada

Para clasificación multiclase, la función de pérdida estándar es la *entropía cruzada* (*cross-entropy loss*) [James et al., 2023, Cap. 4]:

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\log\big(\hat{p}_{ik}\big), \tag{8}$$

donde $y_{ik} \in \{0, 1\}$ es la codificación *one-hot* de la clase verdadera, y $\hat{p}_{ik}$ es la probabilidad predicha para la clase k en la observación i. Las probabilidades se obtienen mediante la función *Softmax*:

$$\hat{p}_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{K}\exp(z_{ij})}, \tag{9}$$

2.2.4 Regresión Logística Multinomial

Para $K > 2$ clases, la codificación Softmax trata las K clases de forma simétrica, asumiendo $K = 1, \ldots, K$ [James et al., 2023, Sec. 4.3]:

$$\Pr(Y = k \mid X = x) = \frac{e^{\beta_{k0}+\beta_{k1}x_1+\cdots+\beta_{kp}x_p}}{\sum_{\ell=1}^{K} e^{\beta_{\ell 0}+\beta_{\ell 1}x_1+\cdots+\beta_{\ell p}x_p}}. \tag{10}$$

La log-odds relativa a la clase base K' es:

$$\log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K' \mid X = x)}\right) = (\beta_{k0} - \beta_{K'0}) + (\beta_{k1} - \beta_{K'1})x_1 + \cdots + (\beta_{kp} - \beta_{K'p})x_p. \tag{11}$$

En PyTorch, `nn.CrossEntropyLoss` integra *log-softmax* internamente por estabilidad numérica, por lo que la capa de salida emite únicamente logits crudos.

2.2.5 Métricas de Evaluación

Para evaluar clasificadores, especialmente con clases desbalanceadas, James et al. [2023, Cap. 4] recomiendan reportar:

- **Accuracy**: de los totales, cuántos datos acertó.
- **Precision**: qué tan seguido ocurre el acierto positivo.
- **Recall**: de los casos reales, cuántos detectó.
- **F1-Score**: media armónica de Precisión y Recall.

2.3 Capítulo 8 – Árboles y Métodos de Conjunto

2.3.1 Árboles de Decisión – Predicción por Estratificación

La predicción mediante estratificación del espacio de características consiste en dividir el espacio de predictores en un conjunto de valores posibles en regiones. Para la observación

en la región R_j, se hace la misma predicción: la media de los valores de respuesta de las observaciones de entrenamiento en R_j.

Los predictores producen rectángulos de alta dimensión por simplicidad e interpretabilidad. Se encuentran los rectángulos $R_1, \ldots, R_J$ que minimizan la Suma de Residuos al Cuadrado (RSS):

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2. \tag{12}$$

Se adopta la **División Binaria Recursiva**: de arriba a abajo, comienza en la cima y va dividiendo sucesivamente, seleccionando el predictor X_j y punto de corte s tal que divida el espacio en las regiones $\{X \mid X_j < s\}$ y $\{X \mid X_j \geq s\}$.

2.3.2 Random Forest y Bosques Aleatorios

Los árboles ensacados aleatorios superan el problema de correlación entre árboles obligando a cada división a considerar solo un subconjunto de los predictores m. Por tanto, en promedio, $(p - m)/p$ no considera al predictor más fuerte, dando oportunidad a otros predictores, decorrelacionando los árboles y produciendo un promedio menos variable.

La diferencia entre árboles aleatorios y ensacados es la elección del tamaño del subconjunto de predictores m. Si se construye un bosque aleatorio usando $m = p$, esto equivale al ensacado. En los datos Heart con $m = \sqrt{p}$, los bosques aleatorios conducen a una reducción tanto en el error de prueba como en el error OOB respecto al ensacado.

Random Forest es especialmente útil con conjuntos de datos biológicos de alta dimensionalidad (por ejemplo, 4 718 genes en muestras de tejido de 349 pacientes con 15 tipos distintos de cáncer).

2.4 Capítulo 10 – Aprendizaje Profundo

2.4.1 Red Neuronal Multicapa (MLP)

Un MLP de L capas ocultas transforma la entrada mediante la recurrencia [James et al., 2023, Cap. 10]:

$$\begin{aligned} \mathbf{h}^{(0)} &= \mathbf{X}, \\ \mathbf{h}^{(\ell)} &= \sigma\big(\mathbf{W}^{(\ell)}\,\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\big), \quad \ell = 1, \ldots, L, \\ \hat{\mathbf{z}} &= \mathbf{W}^{(L+1)}\,\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}. \end{aligned} \tag{13}$$

donde $g(z)$ es una función de activación no lineal, y A_K es una transformación diferente $h_K(X)$.

Función de activación sigmoide:

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}. \tag{14}$$

Función ReLU (preferida en redes modernas):

$$\mathrm{ReLU}(z) = g(z) = (z)_+ = \begin{cases} z & \text{si } z \geq 0, \\ 0 & \text{si } z < 0. \end{cases} \tag{15}$$

James et al. [2023, Cap. 10] destacan que ReLU es más conveniente y que la no linealidad en la función de activación $g(z)$ es esencial: sin ella, el modelo $f(X)$ se reduciría a un modelo lineal en $X_1, \ldots, X_p$. La no linealidad permite capturar no linealidades complejas y efectos de interacción.

Arquitectura MNISTModel (referencia del libro):

- Capa 1: Aplanar (784 entradas) $\to$ Lineal($784 \to 256$) $\to$ ReLU $\to$ Dropout(40%).
- Capa 2: Lineal($256 \to 128$) $\to$ ReLU $\to$ Dropout(30%).
- Salida: Lineal($128 \to 10$).

Modelo con 253 146 parámetros, entrenado 30 épocas con entropía cruzada en 256 observaciones – 168 pasos por gradiente.

2.4.2 Regularización por Dropout

Para controlar el sobreajuste, se aplica *Dropout Learning* [James et al., 2023, Sec. 10.7.3]: durante el entrenamiento, cada neurona se desactiva aleatoriamente con probabilidad p_d:

$$\tilde{h}_j^{(\ell)} = \frac{m_j^{(\ell)}}{1 - p_d} \cdot h_j^{(\ell)}, \quad m_j^{(\ell)} \sim \mathrm{Bernoulli}(1 - p_d). \tag{16}$$

Las redes neuronales son sensibles a la escala de las entradas (Ridge y Lasso también son afectadas). Las entradas con valores de escala de grises de 8 bits entre 0–255 se reescalan al intervalo unitario $[0, 1]$ mediante `ToTensor()`.

2.4.3 Red Neuronal Convolucional (CNN)

Las CNN explotan la estructura local de los datos en cuadrícula mediante operaciones de convolución. Las capas de convolución usan filtros: plantillas que determinan si una característica local particular está presente en una imagen.

Se basan en una operación muy simple llamada convolución (multiplicar elementos de matrices repetidamente y luego sumar resultados):

$$(I * K)_{ij} = \sum_m \sum_n I_{(i+m)(j+n)} \cdot K_{mn}. \tag{17}$$

En una capa de convolución, se usa un banco completo de filtros para detectar bordes y formas orientadas. En las CNN, los filtros se aprenden para la tarea de clasificación específica.

James et al. [2023, Sec. 10.3.2] describen el *MaxPooling* como la operación que reduce la resolución espacial:

$$\mathrm{MaxPool}(I)_{ij} = \max_{(m,n)\in\mathcal{R}_{ij}} I_{mn}. \tag{18}$$

Arquitectura CNN del proyecto (documentada en los apuntes):

Bloque	Capa	Operación	Salida
Entrada	–	Imagen RGB 128×128	$3 \times 128 \times 128$
Conv 1	Bordes, Texturas	Conv(32) + BN + ReLU + MaxPool	$32 \times 64 \times 64$
Conv 2	Texturas, Formas	Conv(64) + BN + ReLU + MaxPool	$64 \times 32 \times 32$
Conv 3	Patrones	Conv(128) + BN + ReLU + MaxPool	$128 \times 16 \times 16$
Clasificador	FC	Flatten(32768) + Linear(512) + Dropout(0.5) + Linear(3)	3 logits

Métricas de evaluación de la CNN:

- **Macro avg**: $\frac{\sum F_1}{3}$ – cada clase es igual.
- **Weighted avg**: pondera F_1 por cuántas fotos tiene cada clase del total. Las clases con más fotos valen más.

3 Metodología: Componente Tabular (MLP)

3.1 Carga y exploración de datos

El archivo `phpnThNfi.arff` fue cargado con `scipy.io.arff.loadarff()`, función que preserva el tipo de cada atributo definido en la cabecera ARFF. Las cadenas de bytes resultantes fueron decodificadas a `str`. El conjunto final contiene 4 839 instancias, 5 variables numéricas de entrada y una variable objetivo binaria (`Class` $\in \{1, 2\}$). La variable objetivo fue codificada con `LabelEncoder` a enteros $\{0, 1\}$, requeridos por la función de pérdida `nn.CrossEntropyLoss`.

3.2 Preprocesamiento

El preprocesamiento siguió dos pasos:

Partición 80/20. Los datos fueron divididos con `train_test_split` de `scikit-learn` usando muestreo estratificado (`stratify=y`), garantizando que la proporción de clases se preserve en ambas particiones. Se obtuvo un conjunto de entrenamiento con 3 871 instancias y uno de prueba con 968.

Normalización (StandardScaler). Las variables de entrada fueron estandarizadas a

media cero y desviación estándar unitaria:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\hat{\sigma}_j}, \tag{19}$$

donde $\bar{x}_j$ y $\hat{\sigma}_j$ son la media y la desviación estándar de la j-ésima variable, calculadas *exclusivamente* sobre el conjunto de entrenamiento. Aplicar la transformación sobre el test usando los parámetros del train evita la *fuga de información* (*data leakage*), práctica explícitamente recomendada por James et al. [2023, Sec. 10.9.1].

3.3 Arquitectura MLP

La arquitectura implementada consta de dos capas ocultas con Normalización de Lotes (*BatchNorm1d*) y Dropout:

$$\underbrace{5}_{\text{entrada}} \xrightarrow{\text{Linear}} \underbrace{256}_{\substack{\text{BatchNorm}\\ \text{ReLU}\\ \text{Dropout}(0.4)}} \xrightarrow{\text{Linear}} \underbrace{128}_{\substack{\text{BatchNorm}\\ \text{ReLU}\\ \text{Dropout}(0.4)}} \xrightarrow{\text{Linear}} \underbrace{2}_{\text{logits}} \tag{20}$$

El modelo posee **35 458 parámetros entrenables**.

3.4 Entrenamiento

El modelo fue entrenado con el optimizador **Adam** [**?**] con tasa de aprendizaje $\eta = 10^{-3}$ y regularización L2 (*weight decay*) $\lambda = 10^{-4}$:

$$\mathcal{L}_{\text{reg}}(\theta) = \mathcal{L}(\theta) + \lambda \sum_{\ell=1}^{L} \|\mathbf{W}^{(\ell)}\|_F^2. \tag{21}$$

Se implementó *early stopping* con paciencia de 20 épocas: el entrenamiento se detuvo en la **época 47**, restaurando los pesos del mejor modelo según la pérdida de validación ($\mathcal{L}^* = 0.02898$). El *scheduler* `ReduceLROnPlateau` redujo η de 10^{-3} a 2.5×10^{-4} en la época 40.

4 Metodología: Componente de Imágenes (CNN)

4.1 Carga del dataset

El conjunto `potato_subset` fue cargado mediante `torchvision.datasets.ImageFolder`, que infiere automáticamente las etiquetas de clase a partir de la estructura de subdirectorios [James et al., 2023, Sec. 10.9.3]. El dataset contiene 456 imágenes RGB en tres clases: *Early Blight*, *Late Blight* y *Healthy*. Se reservó el 20% de imágenes para revalidación, usando lotes de 256 observaciones.

4.2 Transformaciones y aumento de datos

Conjunto de entrenamiento (con aumento de datos):

1. `Resize(128×128)`: redimensionado a resolución fija.
2. `RandomHorizontalFlip(p=0.5)`: volteo horizontal.
3. `RandomRotation(15 grados)`: rotación aleatoria.
4. `ColorJitter`: variación de brillo, contraste y saturación.
5. `ToTensor`: conversión $[0, 255] \to [0, 1]$.
6. `Normalize`$(\boldsymbol{\mu}, \boldsymbol{\sigma})$: normalización por canal con estadísticas de ImageNet ($\boldsymbol{\mu} = [0.485, 0.456, 0.406]$).

Partición 80/20. Se generaron índices aleatorios fijos (`torch.randperm`, semilla 42) dividiendo en 364 imágenes de entrenamiento y 92 de prueba.

4.3 Arquitectura CNN

La CNN sigue la arquitectura de tres bloques convolucionales [James et al., 2023, Sec. 10.3.3]:

$$\underbrace{3\times128\times128}_{\text{RGB}} \xrightarrow{\text{Conv}(32)} 32\times64\times64 \xrightarrow{\text{Conv}(64)} 64\times32\times32 \xrightarrow{\text{Conv}(128)} 128\times16\times16 \xrightarrow{\text{FC}} \underbrace{3}_{\text{logits}} \tag{22}$$

Cada bloque aplica: `Conv2d` → `BatchNorm2d` → `ReLU` → `MaxPool2d(2×2)`. El clasificador: Flatten(32768) → Linear(512) → Dropout(0.5) → Linear(3). El modelo posee **16 872 963 parámetros**.

4.4 Entrenamiento

La CNN fue entrenada con Adam ($\eta = 10^{-3}$, $\lambda = 10^{-4}$) y `ReduceLROnPlateau`. El *early stopping* detuvo el entrenamiento en la **época 28** con $\mathcal{L}^* = 0.12758$.

5 Resultados: Componente Tabular (MLP)

5.1 Curva de pérdida

La Figura 1 muestra la evolución de la entropía cruzada durante el entrenamiento. Se aprecia convergencia rápida en las primeras 30 épocas, la reducción del *learning rate* en la época 40, y la detención por *early stopping* en la época 47. La proximidad entre las curvas indica regularización efectiva.

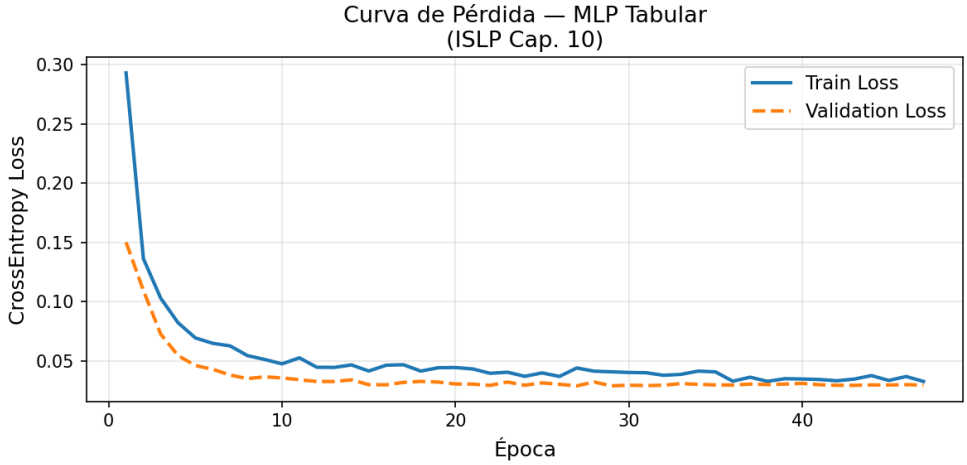

Figure 1: Curva de pérdida (entropía cruzada) del MLP sobre el conjunto de datos tabulares. Línea continua: pérdida de entrenamiento; línea discontinua: pérdida de validación.

5.2 Métricas de evaluación

La Tabla 1 reporta las métricas sobre el conjunto de prueba. Test con 968 patrones reales.

Table 1: Métricas de clasificación del MLP sobre el conjunto de prueba (968 instancias). Se reporta el promedio binario y ponderado dado el desbalance de clases (ratio 17.6:1).

Clase / Agregado	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Clase 1 (mayoritaria, $n = 916$)	—	0.9919	0.9934	0.9927
Clase 2 (minoritaria, $n = 52$)	—	0.9020	0.8846	0.8932
Global (binary avg)	**0.9886**	0.9020	0.8846	0.8932
Global (macro avg)	—	0.9470	0.9390	0.9430
Global (weighted avg)	—	0.9880	0.9886	0.9883

El modelo alcanza un **Accuracy global del 98.86%**. Como advierte James et al. [2023, Cap. 4], en presencia de clases desbalanceadas el Accuracy puede ser engañoso; por ello se reportan también Precision, Recall y F1-Score.

6 Resultados: Componente de Imágenes (CNN)

6.1 Curva de pérdida

La Figura 2 ilustra la evolución de la pérdida y la exactitud por época. La pérdida inicial alta ($\mathcal{L}_1 = 10.44$, típica de pesos aleatorios) desciende bruscamente en las primeras cinco épocas y converge al mínimo de validación $\mathcal{L}^* = 0.1276$ en la época 28.

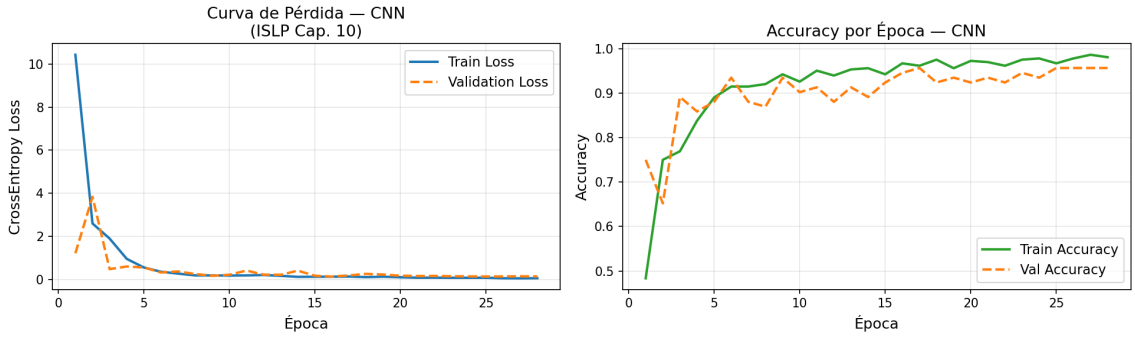

Figure 2: Evolución de la CNN durante el entrenamiento. *Izquierda*: entropía cruzada por época. *Derecha*: accuracy en validación por época.

6.2 Accuracy y matriz de confusión

La CNN alcanzó una **Accuracy global del 94.57%** sobre las 92 imágenes de prueba. La Figura 3 muestra la matriz de confusión en conteos absolutos y normalizada por clase.

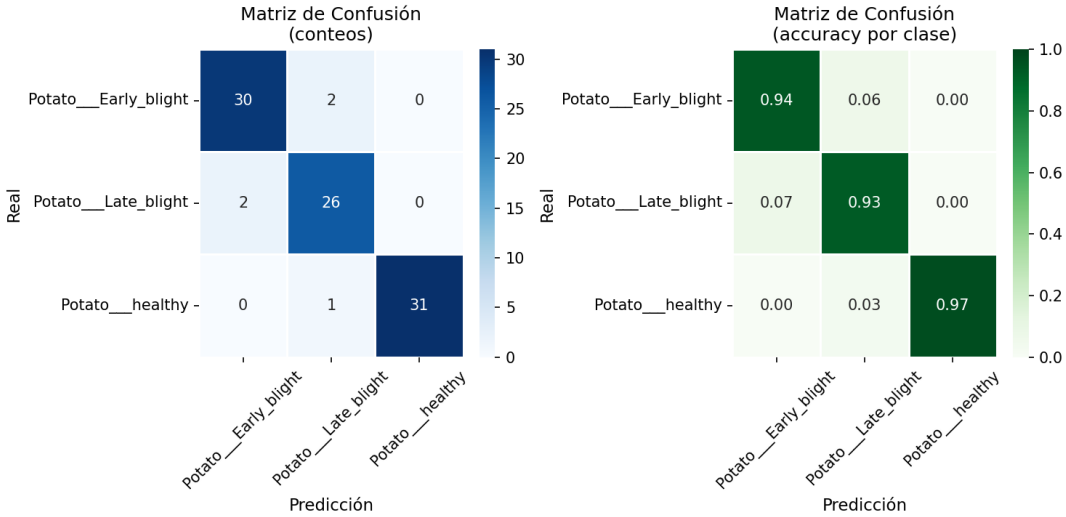

Figure 3: Matriz de confusión de la CNN sobre el conjunto de prueba. *Panel izquierdo*: conteos absolutos. *Panel derecho*: accuracy por clase normalizada por fila.

Table 2: Métricas de clasificación de la CNN por clase (92 imágenes).

Clase	Acc. por clase	Precision	Recall	F1-Score
Early Blight	0.9375	0.9375	0.9375	0.9375
Late Blight	0.9286	0.9000	0.9286	0.9141
Healthy	0.9688	1.0000	0.9688	0.9841
Macro avg	—	0.9458	0.9449	0.9452
Weighted avg	**0.9457**	0.9457	0.9457	0.9453

La clase *Healthy* obtuvo Precision perfecta (1.000): el modelo nunca clasificó una hoja sana como enferma. Las confusiones más frecuentes ocurren entre *Early Blight* y *Late Blight*, enfermedades con síntomas foliares visualmente similares.

7 Análisis y Conclusiones

7.1 Análisis del rendimiento

Importancia de la normalización (Cap. 2 y 10). El MLP tabular se beneficia directamente de `StandardScaler`. Con 5 variables en escalas potencialmente distintas, la ausencia de normalización habría generado gradientes desbalanceados. James et al. [2023, Sec. 10.9.1] enfatizan que normalizar las entradas es un paso obligatorio: las redes neuronales son sensibles a la escala de las entradas (al igual que Ridge y Lasso son afectadas).

Aumento de datos como regularización (Cap. 10). La CNN obtuvo 94.57% con solo 364 imágenes, resultado notable para un dataset reducido. Esto se atribuye al aumento de datos, que James et al. [2023, Sec. 10.3.4] describen como una forma de regularización que enriquece el conjunto de entrenamiento con perturbaciones realistas.

Compromiso sesgo-varianza y early stopping (Cap. 2 y 10). La cercanía entre las curvas de pérdida indica que el *early stopping* ubicó los modelos en el punto óptimo del *bias-variance tradeoff* (Ecuación (5)). Según James et al. [2023, Sec. 10.8], detener el gradiente estocástico antes de la interpolación completa actúa como regularización efectiva.

Desbalance de clases y métricas (Cap. 4). El menor rendimiento del MLP sobre la clase minoritaria (F1 = 0.893 frente a 0.993) ilustra el problema de desbalance de clases. James et al. [2023, Cap. 4] advierten que en estos escenarios el Accuracy global puede ser engañoso, y recomiendan Precision, Recall y F1-Score.

7.2 Limitaciones y trabajo futuro

- El dataset de imágenes (456 imágenes) es pequeño para una CNN entrenada desde cero. El *transfer learning* con modelos preentrenados como ResNet-50 podría mejorar

significativamente el rendimiento [James et al., 2023, Sec. 10.9.4].

- La selección de hiperparámetros fue manual. Una búsqueda sistemática mediante validación cruzada k-fold [James et al., 2023, Cap. 5] permitiría optimizarlos más rigurosamente.
- El desbalance 17.6:1 en los datos tabulares podría mitigarse con SMOTE o ponderación de clases (`weight` en `CrossEntropyLoss`).
- Los árboles de decisión y Random Forest (Cap. 8) constituyen una alternativa interpretable al MLP para el componente tabular, especialmente útil para identificar las variables predictoras más importantes.

7.3 Conclusiones

Este proyecto demuestra que las redes neuronales profundas, implementadas con PyTorch bajo los principios estadísticos de James et al. [2023], constituyen herramientas poderosas para problemas de clasificación en datos tabulares e imágenes. Los factores determinantes en el éxito fueron: (1) el preprocesamiento riguroso (normalización, aumento de datos), (2) la regularización combinada (Dropout, L2, early stopping), y (3) la elección de activaciones ReLU, que mitigan el gradiente evanescente. Los resultados obtenidos validan la metodología y los temas estudiados durante el curso.

References

Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning with Applications in Python*. Springer Texts in Statistics. Springer, Cham, Switzerland, 1 edition, 2023. ISBN 978-3-031-38746-3. doi: 10.1007/978-3-031-38747-0. URL https://www.statlearning.com.